

Smart Project Team Formation Platform

Detailed Explanation of PBL Progress Report – I | Data Structure and Algorithms - II (CCSE0301)

Student	Aman Gupta
Roll No.	2501330100053
Program	B.Tech CSE-A, NIET Greater Noida (AKTU)
Faculty	Mr. Shamshad Ali
Assignment Type	Individual Assignment
Reporting Month	Month 1
SDG Alignment	SDG 4 – Quality Education

1. Project Overview

The project, titled **Smart Project Team Formation Platform**, addresses a common problem in academic settings: students find it difficult to identify suitable teammates for group projects based on skills, interests, and project requirements. Manual team formation is time-consuming and often results in unbalanced teams where skill sets do not complement each other. This progress report documents the first review stage of the project, which was carried out as part of the DSA-II course and maps directly to Unit 1 (Trees) and Unit 2 (Graphs) of the syllabus.

The core idea is to represent student profiles (skills, interests, project area) using tree-based structures for efficient storage and searching, and to represent compatibility between students using graph-based structures, so that suitable teammates can be identified systematically instead of manually. A heap-based ranking layer is also being explored to surface the best matches first.

Aspect	Detail
Core problem	Manual, unsystematic team formation leads to unbalanced skill distribution in project teams.
Proposed approach	Use trees to store/search profiles and graphs to model compatibility between students.
Course mapping	DSA-II Unit 1 (Trees) and Unit 2 (Graphs), extending into Unit 3 in later reviews.
Current stage	Planning and concept-mapping stage; implementation not yet started.
SDG alignment	SDG 4 - Quality Education, by improving collaborative learning outcomes.

2. Problem Understanding

The problem is not simply about storing student information – it is about efficiently *finding the right combination of people* from that stored information. The table below breaks down the understanding of the problem into specific points.

Point	Explanation
Search & comparison	Students should be able to search and compare profiles without manually going through every record.
Key attributes	Skills, interests, project area, and overall compatibility are the main details that decide a good match.
Role of trees	Trees can organize profile data hierarchically, enabling fast search, insertion, and deletion.
Role of graphs	Graphs can model how students are connected or how compatible they are with one another.
Role of heaps	Heaps can help surface the top-ranked matches quickly instead of scanning the full list.
Current stage	The project is at the planning stage; concepts are mapped, but coding/implementation is planned for later reviews.

3. Objectives

Objective	Why it matters
Understand what profile information is useful when forming a project team.	Prevents collecting irrelevant data and keeps the model focused.
Complete study of DSA-II Unit 1 (Trees) and Unit 2 (Graphs), up to adjacency matrix/list.	Builds the theoretical foundation needed before implementation.
Identify where BST and AVL trees help in storing and searching profiles.	Determines which structure balances simplicity and performance.
Use graphs to represent student connections and compatibility.	Forms the backbone of the matching logic between students.
Check whether a heap can be useful for ranking better matches.	Allows quick retrieval of top candidates without full re-sorting.

4. Target Users / Stakeholders

Stakeholder	Role in the System	Benefit
Students	Primary users who create profiles and search for teammates.	Find teammates whose skills complement their own.
Faculty / Project Coordinators	Oversee and approve final project teams.	Reduced manual effort in forming balanced teams.
Project Teams	Formed as an outcome of the matching process.	Get a more balanced combination of skills within the team.
Institution	Provides the academic context for collaborative projects.	Encourages collaborative learning, aligned with SDG 4.

5. Literature Review

Two research papers from 2026 were reviewed to understand existing approaches to student team-formation problems and to identify gaps that this project addresses differently.

Paper 1: Skill Tracker – An Optimization-Based Approach for Automated Student Team Formation in Projects

Authors: Nandini C. Kute, Khushboo A. Prajapati, Balmeet Singh Ramgadia, Prathmesh H. Lohar, Farhat A. Patel (2026).

Paper 2: Dynamic Matching Strategy for College Students’ Innovative Training Projects Based on Reinforcement Learning Optimization

Authors: Xiao Ju, Yumo Ding (2026).

Aspect	Paper 1 (Skill Tracker)	Paper 2 (RL-based Matching)
Core technique	Optimization / fitness-score based matching.	Reinforcement learning based matching.
Use of basic DSA structures	Does not directly use BST, AVL trees, or graph traversal.	Does not focus on basic tree/graph DSA structures either.
Complexity vs. this project	More complex due to resume parsing and wide skill extraction.	More advanced than needed; RL requires training and tuning.
Focus area	Forming teams from a pool of individual students.	Matching students to projects rather than to each other.
Relevance to this project	Confirms the need for a simpler, DSA-driven alternative.	Confirms this project's focus is teammate compatibility, not project allocation.

Overall conclusion: both papers rely on techniques (optimization models, reinforcement learning) that go beyond the scope of a DSA-II course project. This supports the decision to use a simpler, explainable combination of trees (for storage/search) and graphs (for compatibility modeling) in the Smart Project Team Formation Platform.

6. Mapping of DSA-II Concepts to the Project

The table below explains each data structure identified during this review, how it is proposed to be applied in the platform, its time complexity, and why it was chosen over a simpler alternative. These are proposed applications only; none have been implemented yet at this stage.

Concept	Application in this Project	Time Complexity	Why chosen
Binary Tree	Keep student and project records organized in a simple hierarchical structure.	-	Provides a natural hierarchy for grouping records before a search-optimized structure is added.
BST (Binary Search Tree)	Search, insert, or remove a student profile using a chosen key (e.g., student ID or skill score).	Best: $O(1)$ Average: $O(\log n)$ Worst: $O(n)$	Faster than a plain list/array for repeated lookups as the number of student records grows.
AVL Tree	Keeps the profile-storing tree balanced as more students are added.	Best: $O(1)$ Average: $O(\log n)$ Worst: $O(\log n)$	Prevents the worst-case $O(n)$ degradation that a plain BST can suffer from with skewed insertions.
Heap / Max Heap	Brings the strongest matching candidates to the top of the list.	$O(\log n)$ insert/delete	Retrieves the best match instantly without scanning or fully sorting the entire candidate list.
Heap Sort	Arranges compatibility scores or ranked candidates in order.	$O(n \log n)$	Gives a fully sorted ranking list when all candidates (not just the top one) need to be shown.
Tree Traversal	Systematically goes through stored profiles (in-order, pre-order, level-order).	$O(n)$	Needed for listing, exporting, or processing all profiles in a predictable, systematic order.
Graph	Represents students as nodes and compatibility relationships as edges.	-	Naturally models relationships between students, which trees alone cannot represent.
Adjacency List	Stores connections efficiently when the compatibility network is not very dense.	$O(V+E)$ traversal	Saves memory compared to a full matrix when most students are not connected to most others.
Adjacency Matrix	Enables quick pairwise compatibility checks between any two students.	$O(1)$ edge lookup	Useful for small batches of students where instant pairwise lookups matter more than memory use.

Note: All entries above are proposed/possible applications identified during study. None have been implemented in code yet; implementation is planned for the next review stages.

7. Work Completed During Month 1

Activity	Outcome
Finalized the basic idea of the project.	Clear problem statement and project title established.
Identified main users and required profile information.	List of stakeholders and the data fields relevant to matching.
Studied DSA-II Unit 1 (Trees) and Unit 2 (Graphs).	Covered material up to adjacency list and adjacency matrix representations.
Connected tree and graph concepts to the use case.	Mapped BST/AVL/Heap and Graph concepts to specific project needs.
Explored heaps for candidate ranking.	Identified heap and heap sort as possible ranking tools.
Prepared a rough plan for the next stage.	Roadmap drafted for Review 2 (see Section 9).

8. Challenges Faced

Challenge	Explanation
Choosing relevant structures	Deciding which DSA structures genuinely fit this problem, rather than including every structure simply because it appears in the syllabus.
Keeping scope manageable	Keeping the skill and compatibility information simple enough to remain manageable within the project scope.
Balancing usefulness vs. feasibility	Balancing the goal of making the matching idea useful while keeping it realistic to implement within the timeline.
Modeling compatibility as a graph	Understanding how a graph can accurately represent varying degrees of compatibility between students, not just a simple connected/not-connected relationship.

9. Overall Progress

Progress so far: 25%. This covers problem understanding, basic requirement identification, study of the relevant DSA-II topics, review of related research, and initial mapping of DSA concepts to the project. Detailed design and implementation are planned for the next reviews.

10. Plan for Next Review

Planned Task	Purpose
Study Unit 2 and Unit 3 of the DSA-II syllabus.	Builds further theoretical grounding alongside the next review.
Finalize the fields stored in a student profile.	Locks down the data model before implementation begins.

Planned Task	Purpose
Create the first version of the compatibility graph (adjacency list/matrix).	Produces the first working representation of student connections.
Decide which tree and heap structures will actually be used.	Moves from proposed concepts to a concrete implementation plan.
Start a small working version; prepare diagrams/code as evidence.	Provides tangible proof of progress for Review 2.
Improve the matching logic after the first implementation.	Refines accuracy of matches based on real test results.